

ASR: Automatic Speech Recognition Models

From HMM-GMM pipelines to Whisper and wav2vec — 50 years of speech recognition in one session

9 min

Duration

B2 – Understand

Bloom's

L2 – Intermediate

Level

TH-000007

Prerequisite

Session Overview

What you will learn and why ASR is a foundational capability for voice AI

What You Will Learn

- What ASR is and how it works mathematically
- The evolution from HMM-GMM to deep learning ASR
- CTC loss: enabling alignment-free sequence training
- Attention-based encoder-decoder (Listen, Attend and Spell)
- Conformer: CNN + Transformer for state-of-the-art streaming ASR
- Whisper: large-scale multitask ASR
- wav2vec 2.0: self-supervised learning for low-resource ASR
- Word Error Rate (WER) and how to evaluate ASR systems

Concept at a Glance

- Duration: 9 minutes
- Bloom's Level: B2 – Understand
- Difficulty: L2 – Intermediate
- Module: 4.2 Audio and Speech Models
- Prerequisite: Spectrogram and Audio Features (TH-000007)
- Next: TTS Systems (TH-000009)

Learning Objectives

By the end of this session you will be able to:

9 min

Duration

B2 – Understand

Bloom's

L2 – Intermediate

Level

TH-000007

Prerequisite

1

Define Automatic Speech Recognition and state its formal mathematical objective

2

Explain the evolution from HMM-GMM pipelines to modern end-to-end deep learning ASR

3

Describe how CTC loss enables alignment-free sequence-to-sequence training

4

Identify the architectural components of Whisper, Conformer, and wav2vec 2.0

5

Calculate Word Error Rate (WER) and interpret its significance and limitations

6

Select the appropriate ASR model for a given production use case balancing accuracy, latency, and cost

SECTION 1 — THE ASR PROBLEM

What Is Automatic Speech Recognition?

Turning spoken words into text — formally and practically

The ASR Problem: Formal Definition

ASR is a sequence-to-sequence problem: find the most likely word sequence given audio observations

$$W^* = \arg \max_W P(W | X) = \arg \max_W P(X | W) \cdot P(W)$$

W^ = optimal word sequence | X = audio features | $P(X|W)$ = acoustic model | $P(W)$ = language model*

Variability

Same word sounds different across speakers, accents, speaking rates, recording conditions. A model must generalise across millions of voice patterns.

Coarticulation

Sounds blend into each other at natural speaking speed. No clear acoustic boundaries between phonemes — /b/ in 'bad' ≠ /b/ in 'lab'.

Noise

Background music, other speakers, room reverberation overlap with target speech. Real-world WER is 2–5× higher than clean-audio benchmarks.

Long-Range Context

Disambiguating homophones ('bank' financial vs river) requires understanding the full sentence. Local models systematically fail on pragmatic language.

Output options: Plain text, time-aligned words (with timestamps per word), or speaker-diarised segments (who said what when). Modern systems like Whisper produce all three simultaneously from a single forward pass.

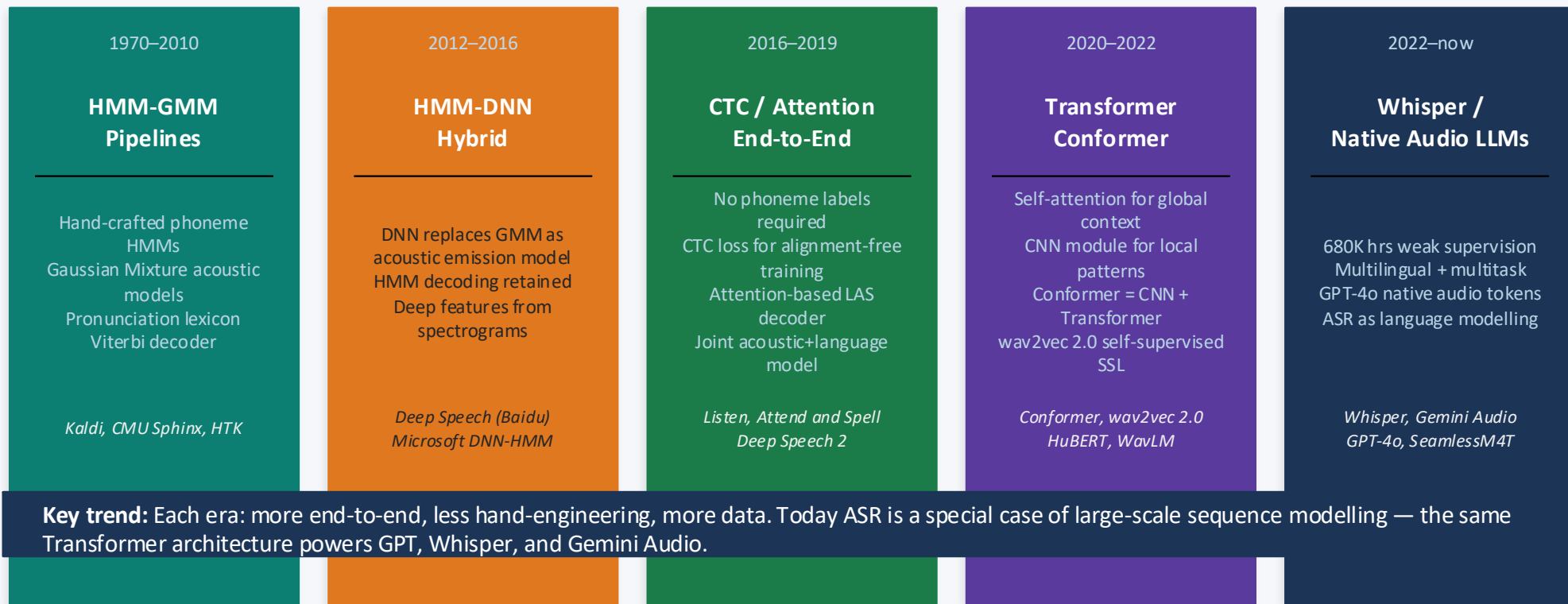
SECTION 2 — EVOLUTION OF ASR

50 Years of ASR: A Visual Timeline

From hand-engineered phoneme models to large-scale neural sequence modelling

The ASR Timeline: Five Eras

Each era brought a fundamental shift in how the problem was approached



Key trend: Each era: more end-to-end, less hand-engineering, more data. Today ASR is a special case of large-scale sequence modelling — the same Transformer architecture powers GPT, Whisper, and Gemini Audio.

Connectionist Temporal Classification (CTC)

The algorithm that enabled end-to-end ASR without frame-level alignment labels

CTC: The Alignment Problem and Its Solution

Audio has 500 frames for 5 seconds; the transcript has 20 characters — how do you train without knowing which frames map to which characters?

The Alignment Problem

Audio: 500 frames | Transcript: 20 characters | No frame-level labels | **Which frames correspond to which characters?**

CTC Solution

1. Introduce a special <blank> token $\emptyset$
2. Output one token per frame (including blanks)
3. Sum over ALL valid alignments that collapse to the target
4. Collapse rule: merge repeated tokens, remove blanks

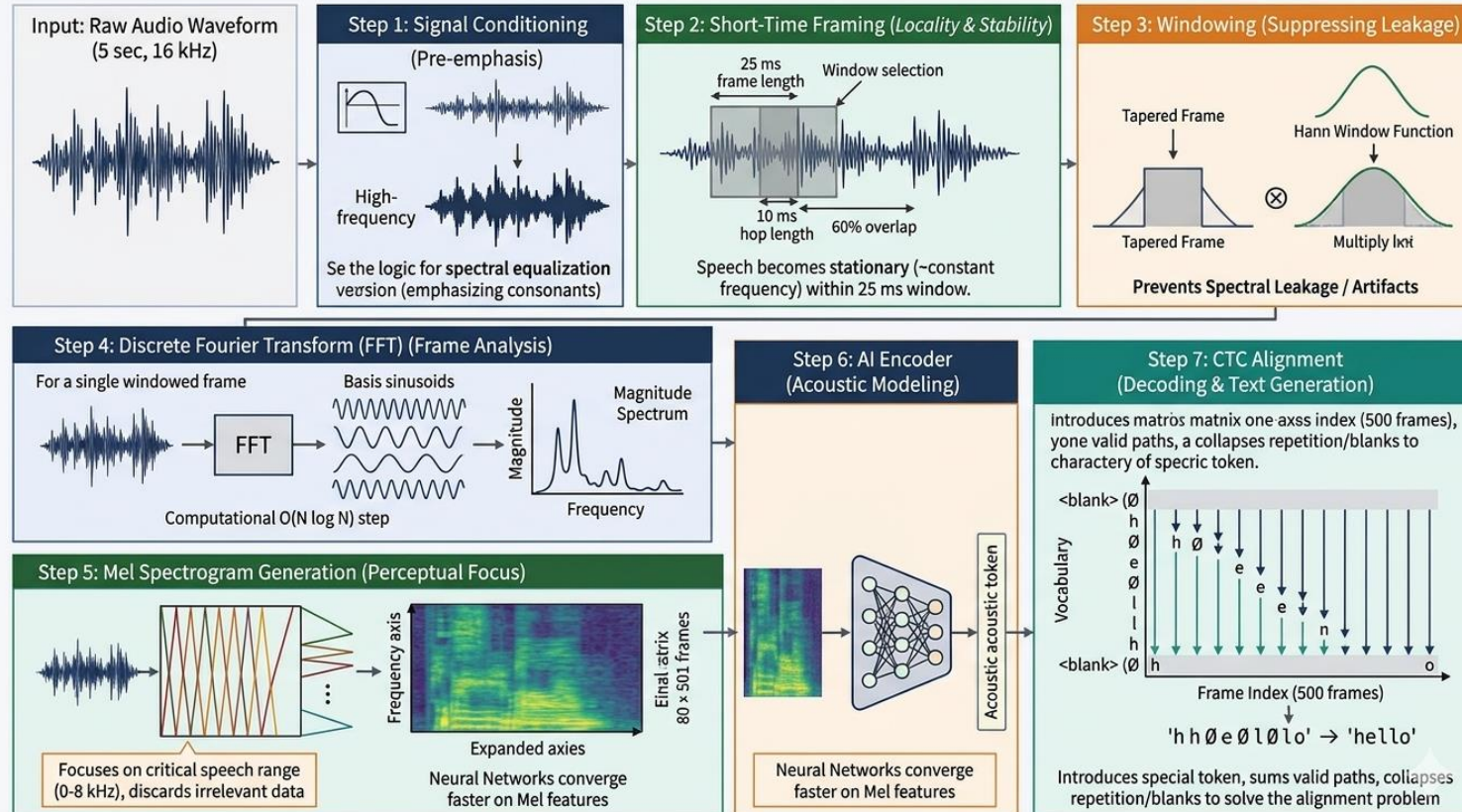
'h h $\emptyset$ e $\emptyset$ l $\emptyset$ l o' $\rightarrow$ 'h e l l o'

Key Properties

- Conditional independence: output at each frame independent given encoder $\rightarrow$ efficient Viterbi decoding
- No need for frame-level phoneme annotations — only text transcripts required
- Streaming-compatible: output tokens incrementally as audio arrives
- Limitation: cannot model output-side language context without a separate LM

Modern use: CTC remains the backbone of streaming ASR (Conformer-Transducer, RNN-T). Google Assistant on Android, YouTube live captions, and Zoom use CTC-based streaming models. Non-streaming (batch) systems like Whisper prefer attention-based decoding.

CTC: The Alignment Problem and Its Solution



Modern ASR Architectures

The two dominant architectures: Conformer for streaming, Whisper for offline multitask

Conformer: CNN + Transformer for the Best of Both Worlds

Pure Transformers excel at long-range dependencies; CNNs excel at local acoustic patterns. Conformer combines both.

Conformer Block (stack 12–18×)

Google, 2020

15.9% relative WER improvement vs pure Transformer on LibriSpeech

Used by:

- Google Assistant
- Android on-device ASR
- YouTube captions

Input: Log-Mel Spectrogram

'Macaron' style: ½ weight on both FFN layers

Feed-Forward (½ weight)
+ Layer Norm

Global context: token i attends to all other tokens

Multi-Head Self-Attention
(relative position encoding)

Local patterns: kernel-based 1D convolution per frame

Convolution Module
(depthwise separable conv)

Residual connection + LayerNorm after each sub-block

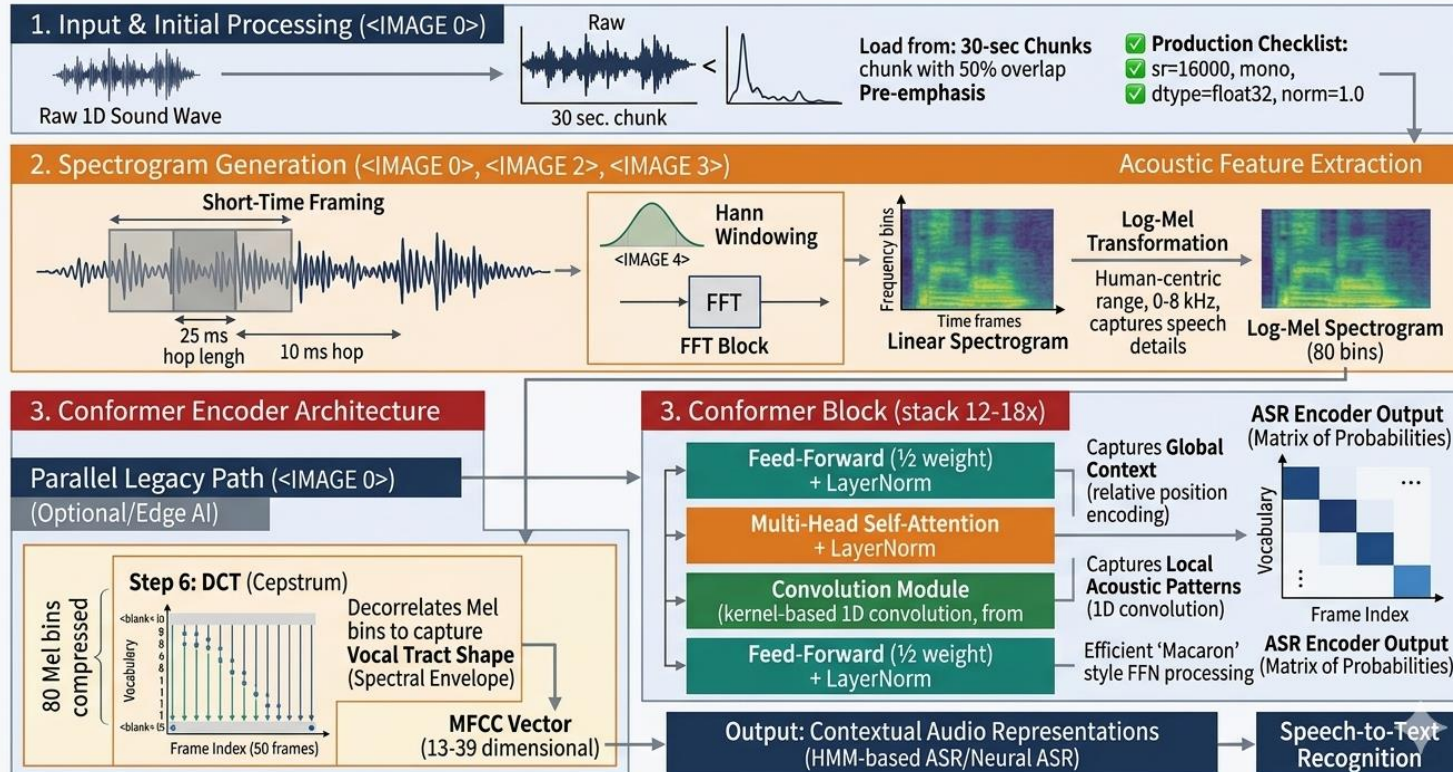
Feed-Forward (½ weight)
+ Layer Norm

Output: Contextual Audio Representations

Streaming variant: Conformer-Transducer (Conformer encoder + RNN-T prediction + joiner). Outputs tokens incrementally as each audio frame arrives. Latency: 200–300 ms. Google, YouTube, Zoom all use this in production.

Conformer: CNN + Transformer for the Best of Both Worlds

Pure Transformers excel at long-range dependencies; CNNs excel at local acoustic patterns. Conformer combines both.



Whisper: Large-Scale Multitask ASR

Radford et al. (OpenAI, 2022) — 680,000 hours of weakly-supervised web audio changed everything

Training Data

680,000 hours of web audio in 99 languages — 100× more than any prior single-dataset approach. Labels are weakly supervised (scraped transcripts, not human-verified).

Architecture

Standard encoder-decoder Transformer. Encoder: 2×Conv1D + Transformer blocks processes 80-bin log-Mel. Decoder: autoregressive Transformer with cross-attention to encoder output.

Multitask Capability

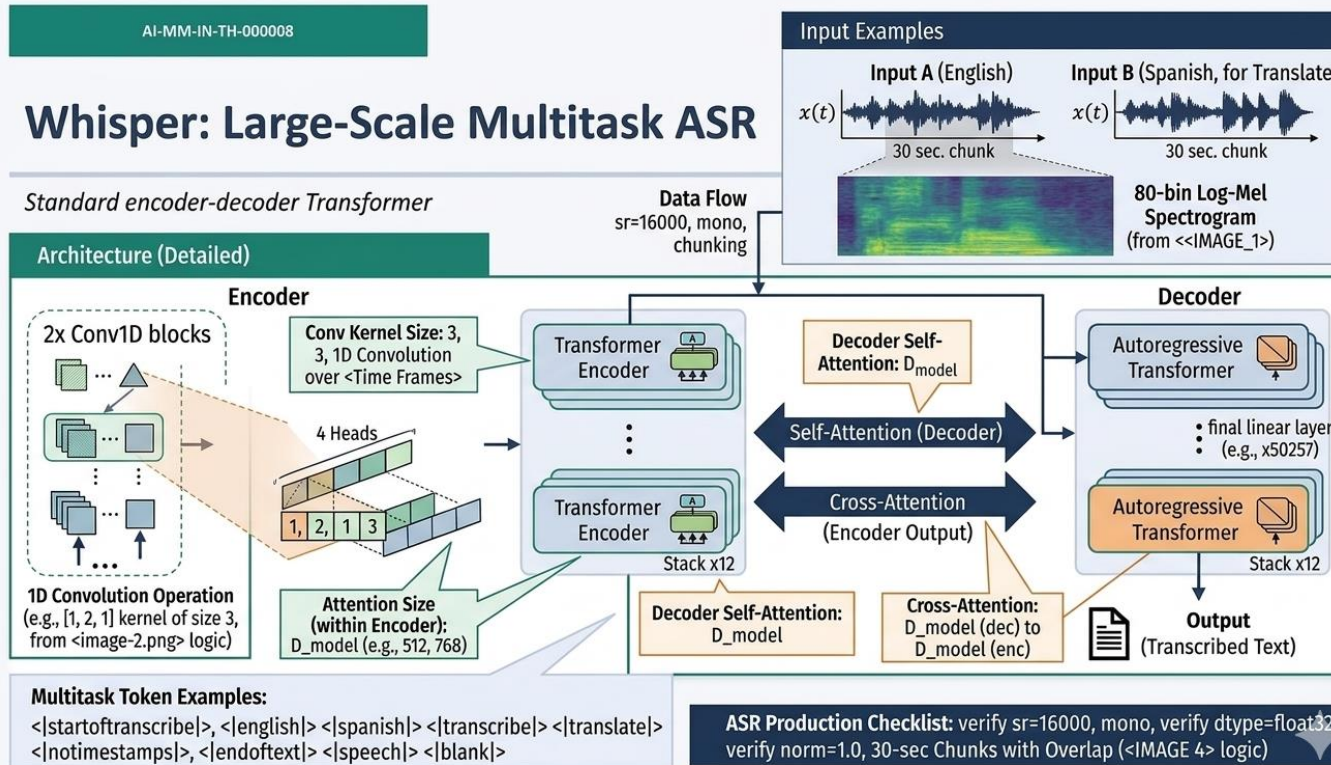
Single model handles: transcription, translation to English, language identification, voice activity detection, timestamp prediction. All controlled by special task tokens prepended to the decoder.

Performance

Whisper Large-v3: 2.7% WER on LibriSpeech test-clean. Matches specialist fine-tuned models without any fine-tuning. Available as API (OpenAI) or self-hosted (faster-whisper, 4–8× faster).

Key insight: Whisper demonstrated that data scale with weak supervision can match carefully curated supervised training. This fundamentally changed how ASR models are trained and deployed — you can now get near-human performance without a single human-verified label.

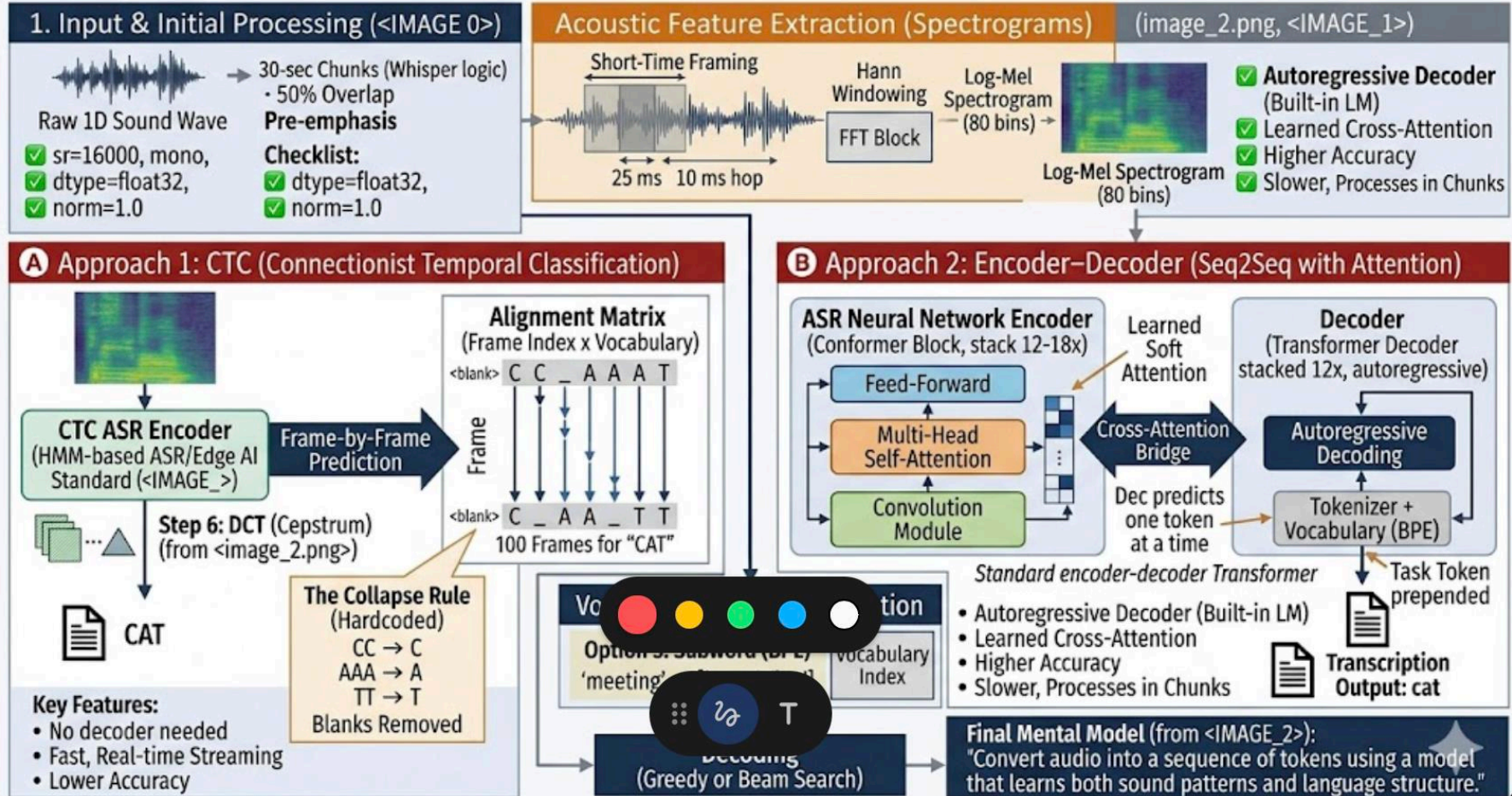
Audio $\rightarrow$ Spectrogram $\rightarrow$ Conv Layers $\rightarrow$ Transformer Encoder $\rightarrow$ Transformer Decoder $\rightarrow$ Text



Each spectrogram frame acts like a token, but the actual token embeddings are learned by the Conv layers (not a lookup table), and positional encoding is still used — just added after the Conv stage instead of at the raw input.

Audio → Spectrogram → Conv Layers → Transformer Encoder → Transformer Decoder → Text

Comparison of ASR Architectures: CTC vs. Encoder-Decoder with Attention



wav2vec 2.0 and Self-Supervised Learning

Achieving near-human WER with 10 minutes of labeled data — the low-resource revolution

wav2vec 2.0: Learning from Unlabeled Audio

Baevski et al. (Meta AI, 2020) — the key idea: learn from 960 hours unlabeled, fine-tune with 10 minutes labeled

Architecture	Training	Result
CNN feature extractor	Raw waveform → latent representations (local, ~20 ms window)	Strong local acoustic features no Mel preprocessing needed
Quantisation module	Latent reps → discrete codes via product quantisation	Converts continuous to discrete (like tokenisation for audio)
Transformer context network	Processes full context bidirectionally via self-attention	Global context modelling over all audio positions
Contrastive loss	Predict correct quantised code for masked frames vs distractors	Self-supervised — no labels needed during 960h pretraining

Results: Fine-tuned with 10 min labeled speech → <5% WER on LibriSpeech. 1 hour → 2.9% WER. Full 960 hrs → 1.8% WER. Why this matters: 99% of world languages have no large labeled speech dataset. wav2vec enables ASR for low-resource languages.

Evaluating ASR: Word Error Rate (WER)

The standard metric for ASR quality — computed as the edit distance between reference and hypothesis at the word level

$$\text{WER} = (S + D + I) / N \times 100\%$$

S = substitutions → wrong word | D = deletions → missing word | I = insertions → extra word | N = total words in reference

Worked Example

Reference: "The quick brown fox"

Hypothesis: "The thick brown fix the"

S=2 (quick→thick, fox→fix), D=0, I=1 (extra 'the')

WER = $(2+0+1) / 4 \times 100\% = 75\%$

Benchmark WER Values

Human parity (LibriSpeech clean): ~2.5–5.5%

Whisper Large-v3 (test-clean): 2.7% WER

Conformer-CTC L (test-clean): 1.7% WER

Real-world (noisy, accented): 2–5× higher than benchmark

WER limitations: WER does not distinguish semantic errors ('one' vs 'won'). CER (Character Error Rate) is used for Chinese/Japanese. BLEU is used for speech translation tasks. Also measure: RTF (Real-Time Factor), latency, memory footprint.

ASR Model Comparison: Choosing for Production

The right model depends on your latency, language, and data requirements

Model	Params	WER (LibriSpeech clean)	Languages	Best Use Case
Whisper Large-v3	1.55B	2.7%	99	Offline, multilingual, zero-shot
Conformer-CTC L	610M	1.7%	English	Enterprise streaming ASR
wav2vec 2.0 XL	600M	1.8%	English	Low-resource fine-tuning
WavLM Large	316M	1.8%	English	SUPERB benchmark tasks
MMS (Meta)	1B	~10%	1,100 langs	Low-resource, multilingual
Parakeet-RNNT (NVIDIA)	1.1B	1.7%	English	NVIDIA GPU streaming

Decision rule: Use Whisper for out-of-box multilingual transcription (no fine-tuning). Use wav2vec/WavLM + fine-tuning for your specific domain with <10 hrs labeled data. Use Conformer-Transducer for real-time streaming (<300 ms latency).

AI-MM-IN-TH-000008 Complete

ASR: Automatic Speech Recognition Models

Advanced Certification in Agentic & Generative AI · IISc Bangalore